

E-COMMERCE CUSTOMER RETENTION & COHORT ANALYSIS

Tutor Instructions

A detailed learning and operating guide for the complete 14-step project

Use this guide to understand the business question, trace each result back to data and code, reproduce the analysis, and explain the project in an interview. It includes practical exercises, answer checks and the boundaries of what the analysis can support.

Evidence base	Edition
UCI Online Retail II; historical UK retail	22 September 2026
Final repository; metric contract 1.1	One combined, executed notebook
14-step implemented analytical scope	English portfolio documentation

Prepared for learning, technical review and portfolio presentation. The Vietnam marketplace context motivates the analytical questions; the observed results describe the historical source retailer.

How to use this guide

Start with the business story before reading SQL. You should be able to explain the unit being counted, the eligible population and the observation period for every headline result. Then use the step chapters to connect the theory to the files.

Reading route	Sections
Orientation	Project brief, scope alignment, repository architecture and environment setup
Guided implementation	Steps 1-14: objective, files, reasoning, acceptance check and exercise
Deeper understanding	Worked examples, single-notebook walkthrough and SQL practice
Independent ownership	Troubleshooting, interview explanations and completion checklist

Recommended learning pattern: read one chapter, open its cited files, explain the calculation in your own words, complete the exercise without copying, and compare with the answer check. A successful pipeline run is useful evidence, but understanding the decisions is the learning objective.

The repository already contains saved notebook outputs, charts, aggregate tables and reports. You can read these immediately. Running notebook cells requires the locally rebuilt database and detailed audit exports. Rebuild from the project root before selecting Run All.

The 14 steps describe the project lifecycle. They do not correspond to 14 executable programs. The full runner executes nine scripts and then one combined notebook. Some planning and reporting steps are curated documents rather than programs.

Document conventions

Paths are relative to the final repository root. Commands assume PowerShell and Python 3.12 on Windows, matching the tested environment. Teaching examples are explicitly synthetic; they are not additional empirical findings. Amounts are GBP gross positive merchandise purchases before credits.

Repository evidence: README.md; reports/project_plan.md; data_dictionary.md

Business purpose and scope alignment

The original proposal framed a fictional Vietnam-focused marketplace inspired by Shopee/Lazada. The purpose was to demonstrate SQL, Python, cohort analysis, segmentation and business reasoning for a data analyst portfolio. The source ultimately selected is a historical UK gift retailer with wholesale activity. Use the marketplace framing as motivation, while keeping findings tied to that retailer.

The main question is: how can an e-commerce team prioritize repeat-purchase, reactivation and customer-service initiatives using observed purchasing history? The intended readers are CRM, growth, commercial managers and technical portfolio reviewers.

Proposal idea	Implemented interpretation
Growth and retention	Monthly KPIs, new/returning buyers, 30/60/90-day repeat purchasing and calendar-month cohorts
Customer value and churn risk	RFM, historical spending concentration, fixed 90-day observed value and inactivity rules
Category, discount and local geography	No native category/discount/city fields; use first-basket value, distinct products and country
Estimated lifetime value	Observed historical and 90-day value; no predictive lifetime forecast
180-day repeat analysis	Listed as an optional extension in the later plan; no published 180-day result
Suggested star schema and Power BI	Invoice/line/customer/activity tables in DuckDB; HTML dashboard; no full prescribed star schema or PBIX

The 14-step implementation is complete under the documented scope. This does not mean every idea in the original proposal was implemented. The original time-to-second-order distribution chart was not delivered separately; conditional timing statistics were delivered. The single notebook follows the final presentation preference.

Repository evidence: [reports/project_plan.md](#); [reports/dataset_selection.md](#); [reports/case_study.md](#)

Repository architecture and data flow

Location	Purpose and how to use it
README.md / data_dictionary.md	Start page and authoritative field/metric definitions. Read before running analyses.
data/raw/	Source instructions and checksum manifest; the original workbook is downloaded locally.
data/processed/	Eleven public aggregate CSVs and their README. Local runs also generate ignored database, Parquet and detailed audit files.
sql/	Seven SQL files covering audit, cleaning, KPIs, behavior, cohorts, RFM and comparisons.
src/	Nine stage scripts, full runner, regression runner, metric_spec.json and product_roles.csv.
src/checks/	Six small regression suites using synthetic fixtures to check edge cases.
notebooks/	One ecommerce_retention_analysis.ipynb with audit/cleaning, cohorts and customer analysis.
images/ / dashboard/	Six PNG charts and an HTML dashboard with aggregate data embedded.
reports/	Planning decisions, stage narratives, summary JSON, final case study and verification evidence.

Follow the dependencies

Original workbook -> raw profile and audited rows -> staged lines -> eligible lines and invoices -> customer features and monthly activity -> KPIs, repeat/cohort and RFM/value/comparison tables -> charts, dashboard and notebook -> business interpretation.

fact_order_lines links to fact_orders by invoice. fact_orders links to customer_features when an ID exists. customer_month_activity collapses multiple invoices into one customer-month. This grain separation prevents an invoice with many items from appearing to be many purchases.

The upload folder is small because large reproducible artifacts stay local. Public aggregate tables are explicitly allowed by .gitignore. Do not upload a virtual environment, cache, temporary files or duplicate database exports.

Repository evidence: data/processed/README.md; .gitignore; sql/01b_clean_data.sql

Environment, execution and first checks

Create the environment from the repository root

```
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m pip check
.\.venv\Scripts\python.exe src/check_project.py
.\.venv\Scripts\python.exe src/run_pipeline.py
```

The first complete run downloads the UCI workbook if absent and verifies its SHA-256. Subsequent runs verify the existing workbook. Internet is needed for initial packages and data acquisition. Local generated data occupy substantially more space than the small public repository; retain free space for the environment and database rebuild.

The pipeline records status, stage times and environment versions in reports/pipeline_run.json. Success requires status passed and a completed notebook stage. The --skip-notebooks option rebuilds scripts only; it is not the full verification route. Keep assertions enabled: do not use -O or a nonzero PYTHONOPTIMIZE setting.

The tested runtime is Python 3.12.14 on Windows. Seven direct dependencies are pinned. Transitive packages are not fully locked, so future resolution and other systems may differ. The earlier audit tested a fresh environment using cached package distributions and a fresh source download; the final packaging run reused a checksum-verified workbook.

Read the notebook and dashboard

```
.\.venv\Scripts\python.exe -m jupyterlab
```

Open notebooks/ecommerce_retention_analysis.ipynb and select Python (retention-analysis). The full runner registers this kernel in its environment. The notebook has 17 code cells. For the dashboard, open dashboard/index.html in a browser with the folder structure intact. GitHub displays HTML source in repository preview; download or clone before opening it locally.

Repository evidence: requirements.txt; src/run_pipeline.py; reports/pipeline_run.json

1. Frame the business problem

Learning objective

Define the decision before selecting charts.

Work through the implementation

Read reports/project_plan.md and write a short brief covering the decision, audience, data scope and evidence needed. Separate the fictional concern about acquisition dependence from observations that the data can actually establish. Identify growth/CRM decisions, commercial questions and measurement needs.

The eight implemented analytical questions span monthly performance, new versus returning customers, repeat windows, cohort behavior, first-basket differences, RFM/value, reactivation and actions. A useful question specifies who would act and which outcome would be monitored.

How to check your understanding

Acceptance: you can connect every analytical output to a question and explain that recommendations have not been deployed. Avoid claiming the retailer has a retention problem solely because a cohort chart declines.

Exercise: turn "customers are not coming back" into a measurable question. Answer check: define a population of first-observed eligible buyers, require 90 days of follow-up, measure the proportion with a second distinct invoice, and compare prespecified groups with the same observation rule.

Repository evidence: reports/project_plan.md; reports/case_study.md

2. Select and evaluate the dataset

Learning objective

Determine which questions the source can answer.

Work through the implementation

Read data/raw/README.md, source_manifest.json and reports/dataset_selection.md. The workbook has two sheets with 1,067,371 item rows spanning December 2009 to December 9, 2011. Available concepts are invoice, product code/description, quantity, timestamp, unit price, customer ID and country.

The download script preserves the original workbook and verifies its hash. The profile script describes feasibility and raw quality. The source is useful because customers recur across time and there are multiple complete monthly cohorts. It lacks native categories, discounts, acquisition channel, delivery experience, margin and Vietnam districts.

How to check your understanding

Acceptance: document provenance, licensing attribution, date coverage, field availability, identity limitations and supported questions. Do not change currency to VND or label countries as HCMC districts to make the data appear local.

Exercise: can this source prove that discounts reduce retention? Answer check: no. There is no customer acquisition discount variable or experimental assignment. Replace that question with a supported first-basket comparison and state its observational nature.

Repository evidence: src/download_dataset.py; src/profile_dataset.py; reports/dataset_profile.json

3. Establish the data dictionary and metrics

Learning objective

Fix definitions before observing desirable results.

Work through the implementation

Use `data_dictionary.md` as the authoritative contract and `src/metric_spec.json` as executable parameters. H is 2009-12-01 inclusive; T is 2011-12-01 exclusive. Primary first-observed cohorts start on 2010-03-01, while earlier purchases remain available for history. Source timestamps retain their wall-clock convention.

Define invoice grain, customer identity, positive merchandise eligibility and gross sales. Record that a first observed order may not be first-ever acquisition. AOV uses invoices; customer retention uses identified accounts. Unknown customers can contribute eligible sales without entering customer analysis.

How to check your understanding

Acceptance: formulas specify numerator, denominator, dates, missing-value treatment and unit. Zero denominators produce null. Future cohort cells are null, while observed no-purchase cells are zero. Metric changes require a contract revision and downstream rebuild.

Exercise: does a buyer making two August orders count as new and returning in August? Answer check: if August is their first observed month, they count once as new and active, and zero as returning; the second invoice can still be a repeat purchase.

Repository evidence: `data_dictionary.md`; `src/metric_spec.json`; `reports/metric_examples.md`

4. Establish the repository structure

Learning objective

Make the analysis understandable and repeatable.

Work through the implementation

Inspect the repository map in README.md. SQL contains business transformations; src contains orchestration and validation; notebooks explain results; reports store interpretation and verification. The current notebook combines the original three sections. Aggregate CSVs now live in data/processed, with no duplicate copies under reports.

Keep source metadata and code in version control. Preserve downloaded originals locally and let the pipeline generate databases and detailed exports. .gitignore allows the eleven named aggregate CSVs while excluding unlisted processed files. requirements.txt describes seven direct package versions.

How to check your understanding

Acceptance: all relative documentation links work, source paths resolve from the repository, and a clean checkout can rebuild. A generated database is not a substitute for documented transformations. Do not place personal paths, credentials or local runtime folders in the public tree.

Exercise: where does a new monthly summary belong? Answer check: generation logic belongs in the relevant src/SQL stage, a public aggregate result belongs in data/processed with a deliberate Git exception, and its interpretation belongs in reports. Avoid manually editing a generated CSV.

Repository evidence: README.md; .gitignore; requirements.txt; data/processed/README.md

5. Audit the original data

Learning objective

Describe defects and exposure before deciding exclusions.

Work through the implementation

Run `audit_data.py` after downloading and profiling. Read `sql/01_data_quality.sql` and `reports/data_quality_report.md`. Preserve every raw row with sheet, Excel-row position and workbook hash. Review missing identity, header conflicts, duplicated lines, worksheet overlap, special product codes, nonpositive values, date coverage and extreme invoices.

Audit findings are not all independent: a line can have several flags. Cross-sheet invoice duplication and repeated lines within one worksheet need separate logic. Match complete normalized invoice payloads as multisets so row multiplicity is preserved. A normal-looking product code does not prove it is merchandise.

How to check your understanding

Acceptance: the original workbook is unchanged, raw count reconciles to 1,067,371 and every source row has lineage. The audit records proposed dispositions and candidate exposure; it does not silently turn provisional amounts into final business metrics.

Exercise: should all duplicate flags be added to estimate deleted rows? Answer check: no. Flags overlap. The final additive waterfall uses one primary disposition per row after explicit precedence. Separate overlap copies from within-sheet duplicate assumptions.

Repository evidence: `src/audit_data.py`; `sql/01_data_quality.sql`; `reports/data_quality_summary.json`

6. Clean and build analytical tables

Learning objective

Convert audited records into explicit populations.

Work through the implementation

Run `clean_data.py`. Read `01b_clean_data.sql` and the versioned `product_roles.csv`. For identical cross-sheet invoice versions, retain the later worksheet; quarantine conflicting versions. Within the retained version, keep the earliest identical line in the primary analysis, and preserve a sensitivity view that retains repeated lines.

Normalize finite prices to six decimals only when deviation is at most $1e-9$ GBP, retaining the raw text and flag. Require numeric invoice ID, positive integral quantity, positive price, valid header/date and merchandise role. Credits and unresolved records remain traceable. Conflicting timestamps or customer headers are quarantined.

Output grains: `stg_order_lines` has every source row; `fact_order_lines` has eligible item lines; `fact_orders` has invoices; `customer_features` has one identified customer; `customer_month_activity` has one active customer-month. Primary output is 972,818 lines, 38,615 invoices and 5,821 identified customers.

How to check your understanding

Acceptance: 972,818 included plus 94,553 excluded equals the raw count. Line/order money reconciles exactly, identifiers and sequences are valid, and sensitivity outputs remain separate. Unresolved exposure of GBP 338,725.57 must remain visible in limitations.

Exercise: why not delete all unidentified invoices? Answer check: valid anonymous sales belong in overall business totals. They cannot support customer-level behavior. This preserves sales while disclosing 87.25% identified-sales coverage.

Repository evidence: `src/clean_data.py`; `sql/01b_clean_data.sql`; `reports/cleaning_report.md`

7. Calculate monthly operating metrics

Learning objective

Build a consistent monthly baseline.

Work through the implementation

Run `monthly_kpis.py` and inspect `02_monthly_kpis.sql`. Start with an observed calendar spine, then aggregate invoices. Orders and gross sales use all eligible invoices; active, new and returning customers use linked customer identities. New means first observed eligible month; returning means an earlier first month.

$AOV = \text{gross merchandise sales} / \text{invoices}$. $\text{Returning-customer share} = \text{returning} / \text{active accounts}$. Identification coverage has invoice and sales versions. Month-on-month growth compares consecutive calendar months; the first month and a zero previous denominator produce null. Empty months retain zero counts and undefined ratios.

How to check your understanding

Acceptance: monthly sales and orders sum to invoice totals; new plus returning equals active for each month; the sum of new customers reconciles to unique historical accounts. Twenty-four complete months are published. The overall AOV is about GBP 490.30, using the full invoice denominator.

Exercise: is returning-customer share a cohort retention rate? Answer check: no. Its denominator is current-month active accounts, whereas cohort retention uses the original cohort size. A high returning share can coexist with weak acquisition or a shrinking customer base.

Repository evidence: `src/monthly_kpis.py`; `sql/02_monthly_kpis.sql`; `data/processed/monthly_kpis.csv`

8. Analyze repeat purchasing

Learning objective

Measure a second invoice with complete follow-up.

Work through the implementation

retention_analysis.py runs the customer-behavior SQL and cohort SQL together. Order identified invoices by timestamp and invoice ID; use the next distinct invoice as the repeat event. Primary cohorts begin in March 2010. The fixed-window eligibility condition is first timestamp + D days < T, with the repeat event at or before that inclusive endpoint.

Exclude incomplete-window buyers from both numerator and denominator, even if an early repeat is visible. Otherwise recent fast repeaters enter the numerator while recent slow buyers have insufficient observation, biasing the rate. Publish own-window populations and the common 90-day population separately.

How to check your understanding

Acceptance: common-population 30/60/90-day results are 690/3,529, 1,158/3,529 and 1,452/3,529. Their rates are 19.55%, 32.81% and 41.14%. The median second-order time is 32.00 days only among the 1,452 qualifying repeaters.

Exercise: a November 1 first order repeats November 2. Is it eligible for 30 days when T is December 1 midnight? Answer check: no. Its inclusive window endpoint equals the exclusive cutoff. Distinct invoices at the same timestamp count in the primary rule; a separate sensitivity collapses those occasions.

Repository evidence: src/retention_analysis.py; sql/03_customer_behavior.sql;
data/processed/repeat_purchase_summary.csv

9. Build cohort retention tables

Learning objective

Separate calendar-month purchasing from elapsed days.

Work through the implementation

Assign each customer to the month of their first eligible observed invoice using all available history. Compute `month_index` from the difference in calendar year/month. Count each customer once per activity month, and divide by the fixed cohort size. Primary heatmap rows begin in March 2010.

Construct the cohort-month grid before joining activity. An observable month with no orders is zero. A future or incomplete target month is null. Month 0 is 100% for a nonempty cohort. The full stored grid has 576 cells, including initial-history cohorts; 276 future cells are unobserved.

How to check your understanding

Acceptance: all counts agree with invoice-derived customer sets, retained count never exceeds cohort size, Month 0 and missing cells follow the contract, and pooled rates weight by eligible cohort sizes. The long CSV supports detailed review; the matrix supports heatmap display; the pooled table uses mature in-scope cohorts.

Exercise: two January buyers return as follows: one in February, none in March, both in April. Answer check: retention is 100%, 50%, 0%, 100%. The increase is valid because the measure describes purchasing in each month, not uninterrupted survival.

Repository evidence: `sql/04_cohort_retention.sql`; `data/processed/cohort_retention.csv`; `data/processed/cohort_retention_matrix.csv`; `data/processed/cohort_retention_pooled.csv`

10. Validate and visualize cohorts

Learning objective

Make visual differences interpretable.

Work through the implementation

Run `visualize_analysis.py` after the customer stage so all required inputs exist. The cohort section in the combined notebook calls independent validation against invoice histories and displays the heatmap and repeat-window chart. Use gray for unobserved future cells and a meaningful scale for observed rates.

Read cohort rows horizontally to follow a cohort as it ages; compare columns to compare similar customer age. Read cohort labels and sample sizes before interpreting color. Calendar seasonality, unequal sample sizes and different acquisition periods can create differences without a changed customer experience.

How to check your understanding

Acceptance: the figure uses the primary cohorts, null values remain masked, and rates and labels agree with exported tables. The common 90-day repeat chart clearly distinguishes elapsed-day windows from calendar-month retention. Small visual differences alone do not establish a causal explanation.

Exercise: what does a blank Month 1 cell for the November 2011 cohort mean? Answer check: December is outside the complete-month reporting period. It does not mean no November buyers returned. Explain this before presenting any heatmap to a manager.

Repository evidence: `src/visualize_analysis.py`; `reports/cohort_visualization.md`; `images/cohort_retention_heatmap.png`; `notebooks/ecommerce_retention_analysis.ipynb`

11. Analyze RFM and customer value

Learning objective

Describe behavior at a fixed snapshot.

Work through the implementation

Run `customer_analysis.py`. R is days since last purchase using full history; F and M use [2010-12-01, 2011-12-01). Keep all 5,821 historical identified accounts, including 1,530 dormant accounts with zero trailing-year F/M and null scores. Among scorable accounts, use midrank scores so ties receive equal scores.

Apply segments in first-match order: Dormant, At Risk Repeat, Inactive Single-Purchase, Champions, Loyal, Promising New, then Recent Occasional. An account with $F \geq 2$ and recency > 90 becomes At Risk Repeat before any spending-based label. Champions require recent repeated activity and high R/F/M scores.

Historical concentration selects $\text{ceil}(10\% \times 5,821) = 583$ accounts. They contribute 63.56% of identified historical gross sales. Fixed 90-day observed value uses the mature primary population and includes the first invoice: mean GBP 682.83, median GBP 369.02. Neither is a predicted lifetime value.

How to check your understanding

Acceptance: segment counts sum to 5,821; trailing-year money and history totals use their own denominators; ties and dormant cases are tested. The 1,728 historical-repeat reactivation candidates are not identical to the 591 At Risk Repeat RFM accounts.

Exercise: can a high-spending account be At Risk Repeat? Answer check: yes, if $F \geq 2$ in the year and recency > 90 . The explicit precedence intentionally prioritizes inactivity over the Champions definition.

Repository evidence: `sql/05_rfm_segments.sql`; `src/customer_analysis.py`; `data/processed/rfm_segments.csv`

12. Investigate business-relevant differences

Learning objective

Check comparability before recommending action.

Work through the implementation

Compare first-order value bands, first-order distinct-product bands and first-order country on the same 3,529 mature primary customers. Freeze attributes at the first invoice to prevent later behavior from redefining the group. Show sample sizes and unadjusted 95% Wilson intervals.

For groups with $n < 100$, retain counts but suppress headline rates and intervals. For cohort standardization, use the same shared cohort months, require at least 20 customers per group-month, then enforce at least 100 in each restricted group. Weight every group using the same pooled cohort weights.

Value-band adjusted rates are withheld because one shared group has only 64 customers; product groups have no qualifying shared months. Geography passes this limited adjustment: UK/other raw rates are 41.10%/41.76%, but standardized rates are 41.77%/39.82%. This reversal cautions against country budget claims.

How to check your understanding

Acceptance: raw and adjusted results are labeled separately; unsupported comparisons remain suppressed; basket value and product diversity overlap; no causal uplift is claimed. The joint basket/product table is descriptive and does not eliminate all confounding.

Exercise: does 46.95% repeat for GBP 250+ baskets prove that increasing basket size will improve retention? Answer check: no. Intent, wholesale purchasing, cohort timing and other factors can differ. Use the association to design a randomized test.

Repository evidence: `sql/06_customer_comparisons.sql`; `data/processed/customer_comparisons.csv`; `data/processed/comparison_by_cohort.csv`

13. Write the case study and recommendations

Learning objective

Translate evidence into a decision and a test.

Work through the implementation

Read reports/case_study.md. Lead with the business question and a few quantified findings, then explain population and method. A finding needs a denominator and an interpretation limit. A recommendation needs an owner, eligible audience, intended action, outcome and guardrails.

Priorities are measurement coverage, a second-purchase experiment, a separate reactivation experiment and service monitoring for large accounts. The second-purchase outcome is intention-to-treat 90-day repeat rate after full observation. The reactivation outcome uses a new clock: purchasing within 60 days of assignment.

How to check your understanding

Acceptance: distinguish existing observations from proposed interventions. Historic customer spending is not recoverable campaign revenue. Set sample size from contemporary eligible baseline, minimum worthwhile effect, power and significance before launch. No numeric lift or return on investment is asserted without an experiment.

Exercise: rewrite "we increased retention by 41.14%." Answer check: "Observed 90-day repeat purchasing was 41.14% (1,452/3,529); the analysis informed a proposed randomized second-purchase trial." The observed baseline is not a treatment effect.

Repository evidence: reports/case_study.md; reports/customer_value_and_segments.md; dashboard/index.html

14. Finalize README and reproduce

Learning objective

Demonstrate that the repository can be independently reviewed.

Work through the implementation

README.md should explain the question, source, method, findings, scope and exact execution steps. Verify links and ensure the public file tree contains code, the single executed notebook, charts, reports and eleven aggregate CSVs. The raw workbook and detailed generated database remain local.

Run the full pipeline and small regression suites, then inspect pipeline_run.json. A notebook must not silently be omitted. The final recorded rebuild completed nine scripts plus the combined notebook; all 17 code cells executed. Six regression suites passed. Public CSVs and charts were unchanged by the packaging reorganization.

How to check your understanding

Acceptance: a new reviewer can read results without executing code and can regenerate all required local inputs using documented commands. Narrative reports are curated; changing data or definitions requires reviewing their prose as well as rerunning computation. A passed run is evidence for the tested workflow, not proof for every environment.

Exercise: replacing an older GitHub folder by copying files can leave which obsolete files? Answer check: the three former notebooks and aggregate CSVs formerly stored in reports. Remove those obsolete copies deliberately while preserving the existing .git folder and unrelated user work.

Repository evidence: reports/reproducibility.md; reports/final_delivery_validation.json; src/check_project.py

Worked examples: grain and observation

The following miniature records are synthetic teaching examples. Use them to test your reasoning before relying on a large table.

Invoice	Customer	Sales	History
A01	A	GBP 30	First observed in August
A02	A	GBP 70	Another August purchase
B02	B	GBP 50	First observed in July
U01	Unknown	GBP 50	No linked identity

August has four invoices, GBP 200 sales and GBP 50 AOV. Active customers = 2, new = 1, returning = 1. Identified orders and sales are both 75%. A six-line A01 invoice remains one invoice. An unknown account is not assigned a synthetic customer ID.

A censored buyer changes the denominator

At an exclusive cutoff of December 1, 2011, suppose A first buys August 31 at 10:00 and repeats exactly 90 days later; B first buys September 2 at midnight and repeats the next day; C first buys September 1 and does not repeat. A and C have complete 90-day windows. B does not, because its inclusive endpoint equals the excluded cutoff. The repeat rate is $1/2$, not $2/3$.

Pool people, not percentages

If two mature cohorts have sizes 2 and 8 and retained counts 1 and 2, pooled retention is $(1 + 2)/(2 + 8) = 30\%$. The unweighted mean of 50% and 25% is 37.5%; it gives the small cohort too much weight for a pooled customer rate.

Tie-aware scoring

For frequencies [1, 1, 2, 5, 5], the contract assigns ascending scores [2, 2, 3, 5, 5]. All-identical values receive score 3. Equal values stay together, so score groups need not contain equal customer counts. Recency reverses the score because lower recency is more recent.

Repository evidence: [reports/metric_examples.md](#)

Read and explain the combined notebook

The single notebook has three original analytical sections and a table of contents. Its saved outputs make it a readable portfolio artifact. It reviews the pipeline outputs and independently validates selected results; it is not a replacement for the download and table-building scripts.

Section	What to trace
Raw audit and cleaning	Read the source summary, missing fields, overlap, price precision, coverage, product codes and cleaned table counts. Connect exclusions to the final waterfall.
Cohort validation	Trace first-observed history, primary population, exact window eligibility and calendar-month activity. Read validation results before viewing the heatmap.
RFM and customer value	Trace the snapshot, score rules, segment precedence, historical concentration, 90-day value, suppression and cohort-mix comparisons.

Run cells top to bottom in the registered environment after the pipeline has created the database. Later sections intentionally reinitialize some variables. Running a late cell by itself can use a stale variable from another section. Restart Kernel and Run All gives a cleaner check than repeatedly executing isolated cells.

For an interview, select one result and follow its lineage: notebook display -> JSON or database table -> Python stage -> SQL transformation -> eligible invoices -> original audited rows. Explain what the independent validation reconstructs and why it can catch a grouping or boundary error.

Practice SQL on the rebuilt database

```
SELECT COUNT(*) AS invoices,  
       SUM(order_sales_gbp) AS gross_sales_gbp  
FROM fact_orders;
```

```
SELECT COUNT(DISTINCT customer_id) AS customers  
FROM fact_orders WHERE customer_id IS NOT NULL;
```

Expected values are 38,615 invoices, GBP 18,933,017.227 before display rounding and 5,821 identified accounts. These are read-only practice queries. Some production SQL depends on parameters and registered helpers supplied by its Python runner, so run the full stages through their documented entry points.

Repository evidence: notebooks/ecommerce_retention_analysis.ipynb; sql/02_monthly_kpis.sql

Troubleshooting and safe updates

Symptom	Likely cause and response
Module import fails	Use the .venv Python explicitly. Install requirements and run pip check. A different Jupyter kernel can use a different environment.
Database/table missing	Saved outputs are readable, but executing cells needs a rebuild. Run <code>src/run_pipeline.py</code> from the repository root.
Database locked	Close notebook/database connections and retry the relevant stage. Do not delete source data to solve a lock.
Source hash mismatch	Stop and inspect the source/version or incomplete download. Keep the manifest check; do not disable it to force success.
Missing required notebook	Restore <code>ecommerce_retention_analysis.ipynb</code> . The preflight guard deliberately refuses an incomplete checkout.
Pipeline failed	Read <code>failed_step</code> and error in <code>reports/pipeline_run.json</code> , fix the cause, and rerun. A partial run is not a validated release.
PDF/report and regenerated numbers differ	Narrative reports are curated. Review all dependent text after changes to source, mapping, dates or definitions.
HTML appears as code on GitHub	Clone/download the folder and open <code>dashboard/index.html</code> locally. It is not automatically a hosted website.

When changing definitions, update the metric contract and executable parameters together, revise affected SQL/Python, rerun checks and the full pipeline, reconcile outputs, and then update narrative interpretation. A different cutoff changes maturity, RFM, geography comparisons and customer-value denominators together.

Keep the original workbook unchanged and preserve evidence for exclusions. Never manually edit an output table to make a headline match a preferred number. When publishing an update, review the actual Git file list and retain only intended public artifacts.

Repository evidence: `src/run_pipeline.py`; `reports/pipeline_run.json`; `.gitignore`

Interview practice and learning sign-off

A concise project explanation

"I built a reproducible SQL and Python workflow to analyze repeat purchasing and customer value in more than one million historical retail transaction lines. I documented cleaning decisions, created invoice and customer tables, validated cohort and RFM calculations, and translated the findings into testable CRM recommendations. The observed 90-day repeat rate was 41.14% for 3,529 fully observed customers. The source is historical UK retail, so I would validate the approach on current local data before an operational rollout."

Question	A strong answer includes
Why a 90-day denominator?	Everyone receives complete follow-up; even an early repeater is excluded if their full observation window is incomplete.
Why can cohort retention rise?	A customer can skip a month and buy again; this is monthly purchasing, not continuous survival.
Why keep unidentified sales?	Overall sales and customer analytics have different identity requirements, with linkage coverage disclosed.
Why no discount recommendation?	The source lacks the field. Basket associations support hypotheses, not causal promotion claims.
Why 1,728 versus 591 at risk?	Reactivation uses historical repeat frequency and includes dormant repeaters; the RFM label uses trailing-year frequency and segment precedence.
What would you change for Vietnam?	Acquire appropriate contemporary local data, preserve lineage, define local business rules, and revalidate metrics and experiments.

Learning sign-off: you can reproduce the project, explain all headline denominators, justify the main cleaning decisions, distinguish observation from prediction and causality, identify scope deviations, and modify one parameter while describing every downstream consequence. Do not claim independent authorship of work you cannot explain.

Suggested progression: first explain results without code; next trace one result through SQL and Python; then solve the synthetic examples unaided; finally rebuild and present a five-minute business recommendation with limitations.

Repository evidence: [reports/case_study.md](#); [reports/metric_examples.md](#); [reports/reproducibility.md](#)